

IS 201 TA Lab



James' Super Cool
IS 201 Cheat Sheet

You

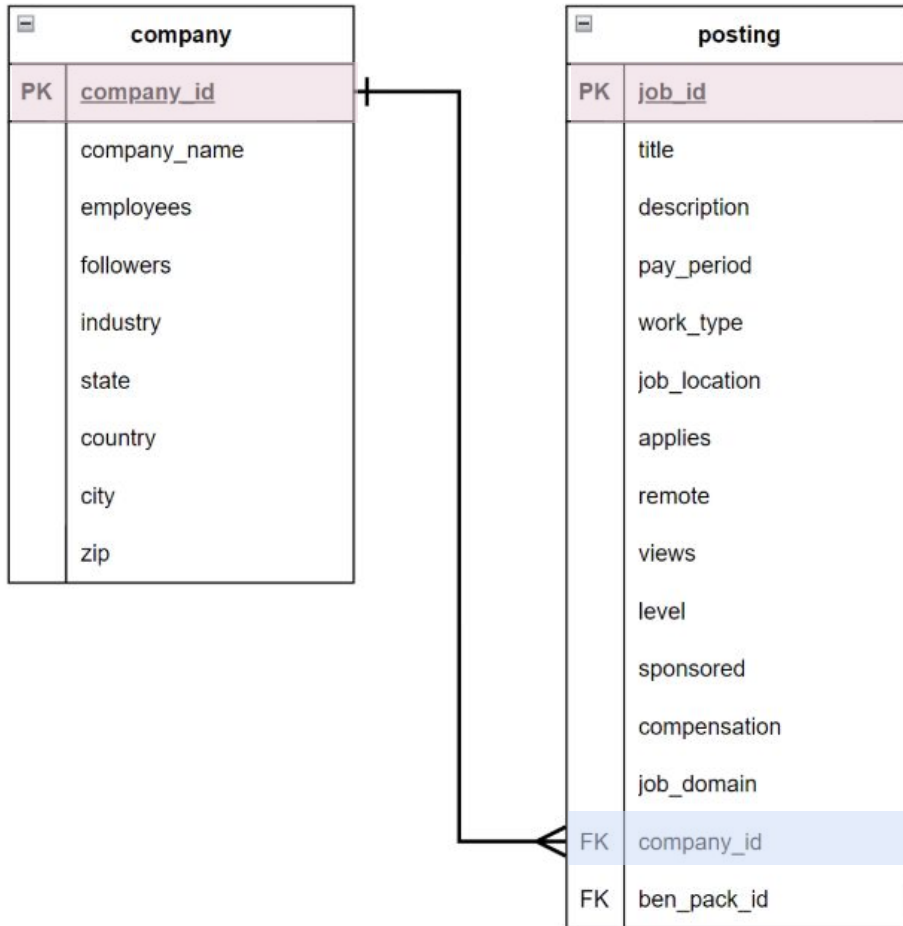


bit.ly/4tZYPBK

INDEX

Slide	Topic
1	ERDs
7	SQL
9	Flow Charts
10	VBA
16	Statistics
17	Tableau
18	Solver
19	HTML
28	CSS

ERD: PRIMARY/FOREIGN KEYS

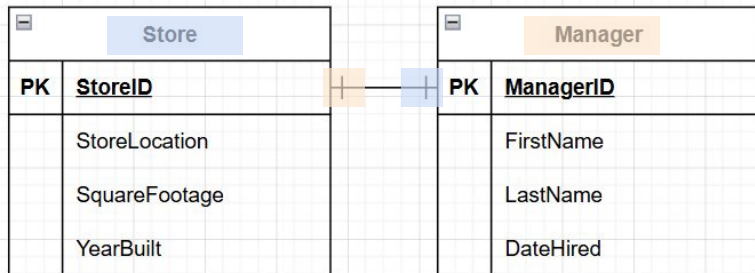


A **primary key** is a unique identifier (think social security number)

A **foreign key** is another table's primary key, used to link two tables together

Entity
Relationship
Diagram

ERD: CARDINALITY

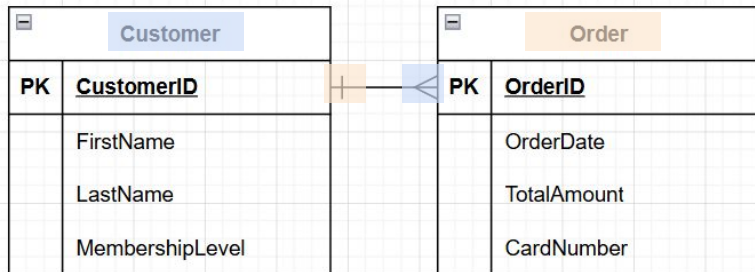


Read left-to-right AND right-to-left

One to One (1:1)

A store has one manager

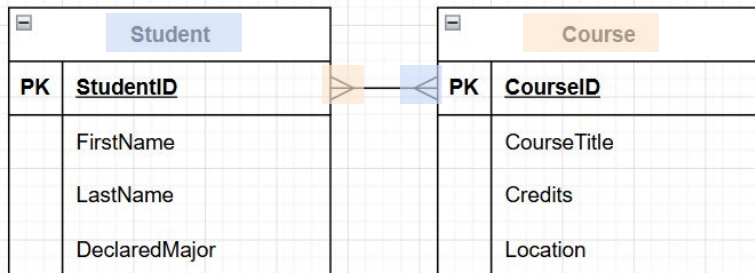
A manager works at one store



One to Many (1:N)

A customer can have multiple orders

An order belongs to one customer



Many to Many (M:N)

A student can enroll in many courses

A course can have lots of students

Note! M:N cardinality requires a composite table (next slide)

ERD: COMPOSITE TABLE

movie	
PK	<u>movieid</u>
	title
	mpaa_rating
	budget
	gross
	release_date
	genre
	runtime
	rating
	rating_count
	summary

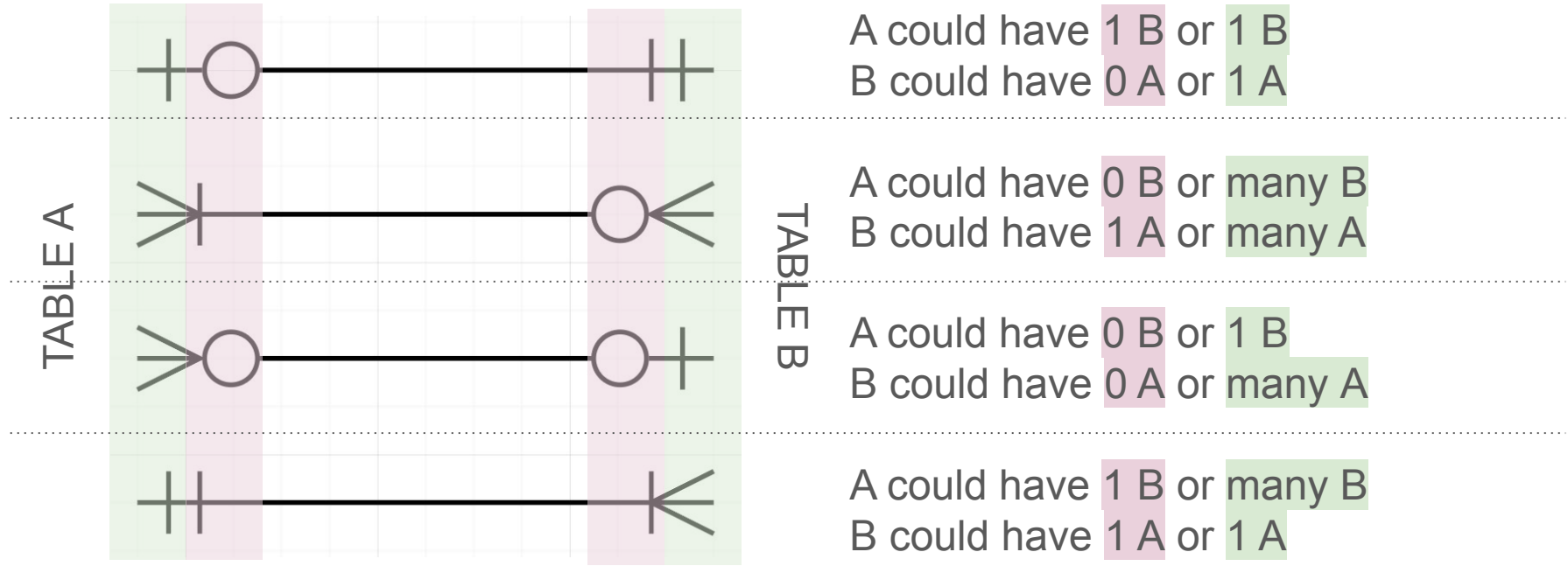
character	
PK/FK	<u>movieid</u>
PK/FK	<u>actorid</u>
	character_name
	credit_order
	pay
	screentime

actor	
PK	<u>actorid</u>
	name
	date_of_birth
	birth_city
	birth_country
	height_inches
	biography
	gender
	ethnicity
	networth

To know how much money Tom Hanks got **paid** to play Woody in Toy Story, you need to know both the **movie** and the **actor**. His pay is probably different than when he was Woody in Toy Story 2, or when he was Forrest Gump in Forrest Gump

ERD: MINIMUM/MAXIMUM CARDINALITY

The inner ones are minimum (0 or 1)
and the outer ones are maximum (1 or many)



Think: A student could have 0 cars, but they could also have multiple

SQL

```
1  SELECT (DISTINCT) -- attributes
2  FROM -- tableA
3  JOIN -- tableB ON tableA.attribute = tableB.attribute (order doesn't matter)
4  WHERE -- attribute filters [=, !=, <>, IS, LIKE '%__%', IN ("__", "__", "__")]
5      AND/OR -- you only say WHERE one time, but can have many filters
6  GROUP BY -- all non-aggregated attributes when aggregating
7  HAVING -- group-based filters (not used in the SQL project)
8  ORDER BY -- attributes in ASC (default) / DESC order
9  LIMIT -- to xx top results
```

10

```
11  -- aggregate(attribute) AS newName
```

```
12  count()
```

```
13  avg()
```

```
14  min()
```

```
15  max()
```

```
16  etc.
```

Structured
Query
Language

SQL: WHERE USE CASES

ta_name = "James" → Exact match

ta_name != "James" → Not exact match

ta_name <> "James" *(these are equivalent)*

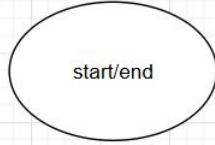
ta_name **IS** NULL → NULL means blank data (can also do **IS NOT** NULL)

ta_name **LIKE** '%ame%' → The text ("ame") is contained within the attribute
(% = wildcard)

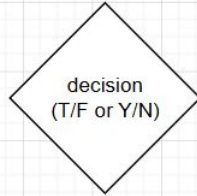
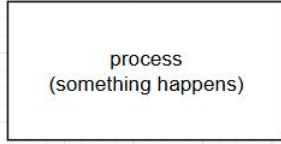
ta_name **IN** ("James", "Robert", "Frankie")
→ Exact match for any of these

FLOW CHARTS

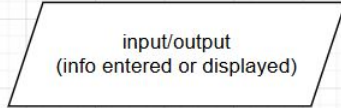
go to
draw.io



you can only have ONE start
but you CAN have multiple ends



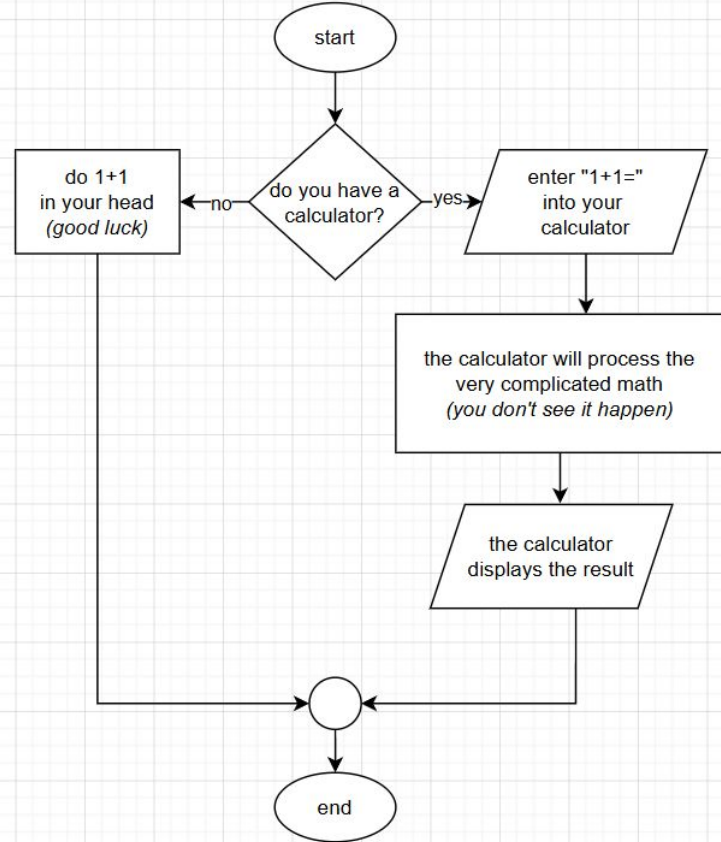
decisions are the ONLY thing that
can have more than one arrow
pointing OUT of them



connectors are the ONLY thing that
can have more than one arrow
pointing INTO them

file > export as > pdf

EXAMPLE:
How to solve 1+1



VBA: BASICS

Option Explicit 'This makes it so that you can only use variables you've declared (VERY RECOMMENDED)

```
Sub thisIsMySubName()
```

```
    'BASICS!
```

```
    'Generally you will first refer to an object type (like a Sheet)
```

```
    'and then tell it what you want to do (like delete or add or copy, etc):
```

```
        ' Sheets and Worksheets are the same thing
```

```
        Sheets("MySheet").Delete
```

```
        Sheets.Add.name = "MySheet"
```

```
        Worksheets("MySheet").Activate
```

```
        ' Range and Cells are SIMILAR but different: Range("E7") = Cells(7, 5)
```

```
        Range("A1").Activate
```

```
        ActiveCell.Value = "Hello"
```

```
        Range("A1:D3").Copy
```

```
        Range("E5").PasteSpecial
```

```
        ' Columns and Rows are what they sound like
```

```
        Columns("B:D").Delete
```

Visual
Basic
for
Applications

VBA: NAVIGATION AND MISC.

'NAVIGATION!

```
'
ActiveCell.End (xlDown)
ActiveCell.End (xlToLeft)

' Move (rows, columns) based on a cell or range
ActiveCell.Offset(1, 2).Value = "NewSpot"
    Range("A1").Offset(3, 4).Value = "NewSpot" 'The cell that now says "NewSpot" is E4
```

' MISCELLANEOUS!

```
' Bolding
Range("A1").Font.Bold = True

' Autofitting columns
Columns.AutoFit 'all columns
Columns("A:C").AutoFit 'specific columns

' Currency format
Range("A1").Style = "Currency"

' Bottom border
Range("A1").Borders(xlEdgeBottom).LineStyle = xlContinuous

' Text splicing
name = "James"
first_initial = Left(name, 1)
```

VBA: VARIABLES

```
'DECLARE VARIABLES!
```

```
Dim i As Integer    'a whole number
Dim j As Double     'a number with a decimal
Dim k As String     'a line of text
Dim l As Boolean    'true or false
Dim m As Worksheet
```

```
i = 5
j = 1.2345
k = "Hello there! General Kenobi!"
l = True
Set m = Sheets("Sheet1") 'objects (sheets, books, ranges) use set instead of =
```

```
'USER INTERFACE!
```

```
'Tell the user something
MsgBox "Oh hello there user, welcome to my spreadsheet"
```

```
'Ask for an input
Dim username As String
username = InputBox("What's your name?")
```

```
' COMBINING STRINGS
```

```
Dim myOutput As String
Dim name As String
name = "James"
```

```
' Combine strings with ampersands (&)-- don't forget spaces
myOutput = "Hello. My name is " & name & " and this is Disney Channel!"
```

'CONDITIONALS!

VBA: CONDITIONALS

```
' If Statements
If i < 0 Then
    ' your code here
ElseIf i < 5 Then
    ' your code here
ElseIf i < 10 Then
    ' your code here
Else
    ' your code here
End If
```

```
'Case Statements
Select Case fruit
    Case "apple", "strawberry"
        Color = "red"
    Case "grape"
        Color = "purple"
    Case Else
        Color = "unknown"
End Select
```

```
'Another Case Statement
Select Case temperature
    Case Is >= 100
        feel = "too hot"
    Case 80 To 100
        feel = "pretty warm"
    Case Is < 80
        feel = "I'm probably staying out of the pool"
End Select
```

'LOOPS!

VBA: LOOPS

```
'For Loop
For i = 0 To 5
    ' your code here
Next i
```

```
'For Each Loop
Dim x As Range
For Each x In Range("A2", Cells(lastrow, "A"))
    ' your code here
Next
```

```
'For Each Loop 2
Dim x As Range
Dim mtRange As Range
Set myRange = Range("A1:E5")
For Each x In myRange
    ' your code here
Next
```

```
'Do While Loop
Do While i = 5 'condition does NOT have to be numbers
    ' your code here
Loop
```

```
' Do Until Loop
Do Until i = 10 'condition does NOT have to be numbers
    'your code here
Loop
```

End Sub

VBA: FUNCTIONS

```
Function thisIsMyFunctionName(num As Integer, tf As Boolean) As String
```

```
' a function declaration is just like a set of dims  
' the INPUT is in the parentheses ()  
' the OUTPUT is outside of it  
' so this function KIND OF has this baked into it:
```

```
Dim num As Integer
```

```
Dim tf As Boolean
```

```
Dim thisIsMyFunctionName As String
```

```
' the INPUT then comes from your excel sheet  
' for example, in your sheet you cell might say:  
' =thisIsMyFunctionName(15, True)
```

```
' the OUTPUT comes from your function  
' set the function's name equal to something  
' in this case, our output has to be a string:
```

```
thisIsMyFunctionName = "That's a very nice hat you have there, partner!"
```

```
' generally your inputs will be used to calculate/decide your output
```

```
End Function
```


Correlation

numeric - numeric
pearson r coefficient

Regression

numeric - multi. numeric
R²

ANOVA

numeric - mult. categorical
p-value

T-test

numeric - 2 pair categorical
p-value

STATS

r: $\overset{-1}{\text{As } x \uparrow, y \downarrow} \overset{0}{\text{As } x \uparrow, y \uparrow} \overset{1}{\text{As } x \downarrow, y \uparrow}$

R^2 : 0 ————— 0.7 ————— 1

^

70% of the variance
in y is explained by x

p: 0 _____ α _____ 1

usually 0.05
v

Reject the null Fail to reject the null

Statistically significant Not

Null Hypothesis

H_0 : No x's are significant

Alt Hypothesis

H_A : At least one x is significant

Scientific

Notation:
 $a \mathbf{E} b = a \cdot 10^b$

$$\begin{aligned} \mathbf{aE3} &= \mathbf{a} * 10^3 \\ &= \mathbf{a} * 1000 \text{ (big)} \end{aligned}$$
$$\begin{aligned} \mathbf{aE-3} &= \mathbf{a*10^{-3}} \\ &= \mathbf{a*0.001} \text{ (small)} \end{aligned}$$

TABLEAU

Things to mention:

“Default”

Switching x and y axis, ordering

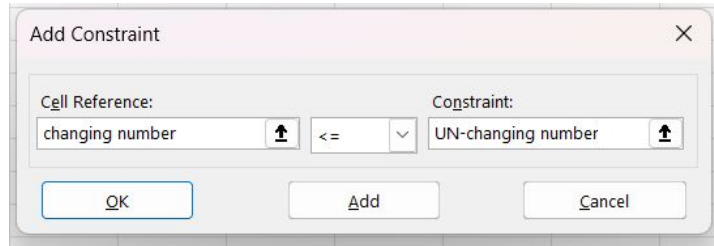
SOLVER

Two Important Excel Formulas:

=sum(A1:A5) $\rightarrow$ A1 + A2 + A3 + A4 + A5

=sumproduct(A1:A5, B11:B15) $\rightarrow$ A1*B11 + A2*B12 + ... + A5*B15
the two ranges should be the same size and shape

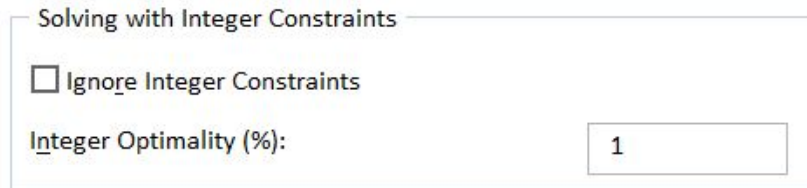
Constraints:



The screenshot shows the 'Add Constraint' dialog box. It has a title bar with a close button. Inside, there are two input fields: 'Cell Reference:' and 'Constraint:'. The 'Cell Reference:' field contains the text 'changing number' and has a small icon to its right. The 'Constraint:' field contains the text 'UN-changing number' and also has a small icon to its right. Between these two fields is a dropdown menu showing '<='. At the bottom of the dialog are three buttons: 'OK', 'Add', and 'Cancel'.

IF INTEGER ISN'T WORKING

Solver > Options > uncheck this:



The screenshot shows the 'Solving with Integer Constraints' section of the Solver Options dialog. It contains a checkbox labeled 'Ignore Integer Constraints' which is currently unchecked. Below this is a field labeled 'Integer Optimality (%)' with the value '1' entered.

HTML: SETUP

In an html file inside VS Code, type an exclamation point and enter to generate:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

Hyper
Text
Markup

Language

startbootstrap.com/themes/portfolio-resume

HTML: TAGS

```
<!-- This is a comment -->
```

```
<h1>Main heading</h1>
```

```
<h2>Section heading</h2>
```

```
<h3>Subsection</h3>
```

```
<p>This is a paragraph.</p>
```

```
<!-- Line Break -->
```

```
<br>
```

```
<tag>
```

stuff

```
</tag>
```

The start and end tags act like parentheses

```
<!-- Unordered List (Bullets)-->
```

```
<ul>
```

```
  <li>Item 1</li>
```

```
  <li>Item 2</li>
```

```
  <li>Item 3</li>
```

```
</ul>
```

```
<!-- Ordered List (Numbers)-->
```

```
<ol>
```

```
  <li>Item 1</li>
```

```
  <li>Item 2</li>
```

```
  <li>Item 3</li>
```

```
</ol>
```

HTML: ANCHORS

```
<!-- Hyperlinks -->
<a href="https://www.google.com">Visit Google</a>

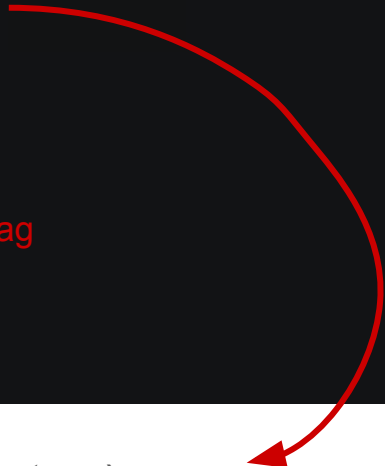
<!-- Folder References -->
<a href="otherpage.html">Visit Other Page</a>

<a href="subfolder/page.html">Visit Subfolder Page</a>

<a href="../yetanotherpage.html">Visit yet another page</a>

<!-- On-page Anchors -->

<a id="section1">Section 1</a>
<a href="#section1">Go to Section 1</a>
```



← any tag, not just a
(in fact it would rarely if ever be an a tag)

../ means go to the parent folder (or *root directory*)

HTML: IMAGES

```
<!-- Images -->  
  
  

```

Notice: images do NOT have an end tag `</img>`
(neither do line breaks `</br>`)

HTML: DIVISIONS

Divs don't inherently do anything: they're invisible boxes you put content into so that you can isolate it for positioning and CSS styling using classes.

HTML:

```
<div class="square-image">
  
</div>
```

CSS:

```
.square-image {
  width: 200px;
  height: 200px;
}
```

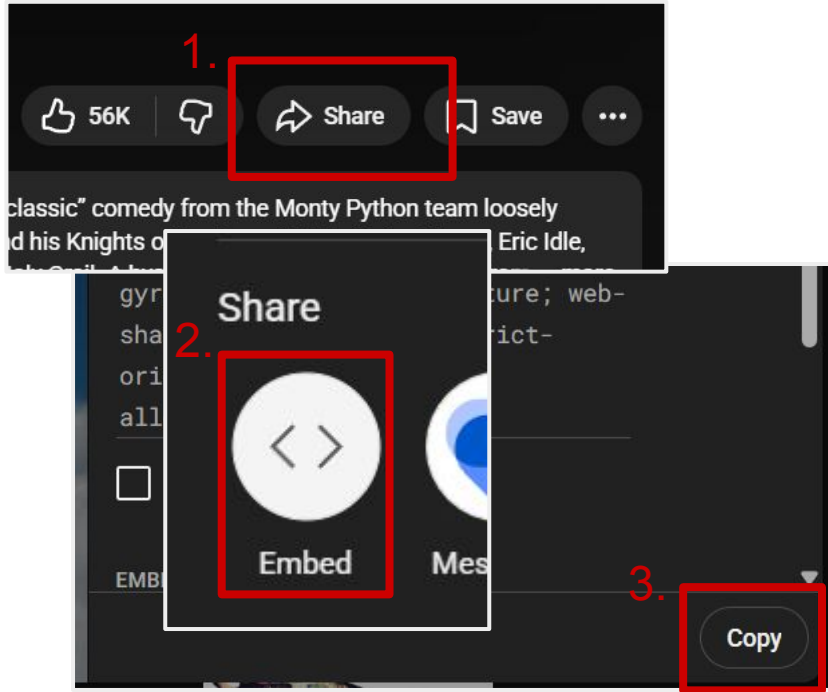
For more, see:

CSS: SELECTORS/PROPERTIES/VALUES

HTML: EMBEDDING

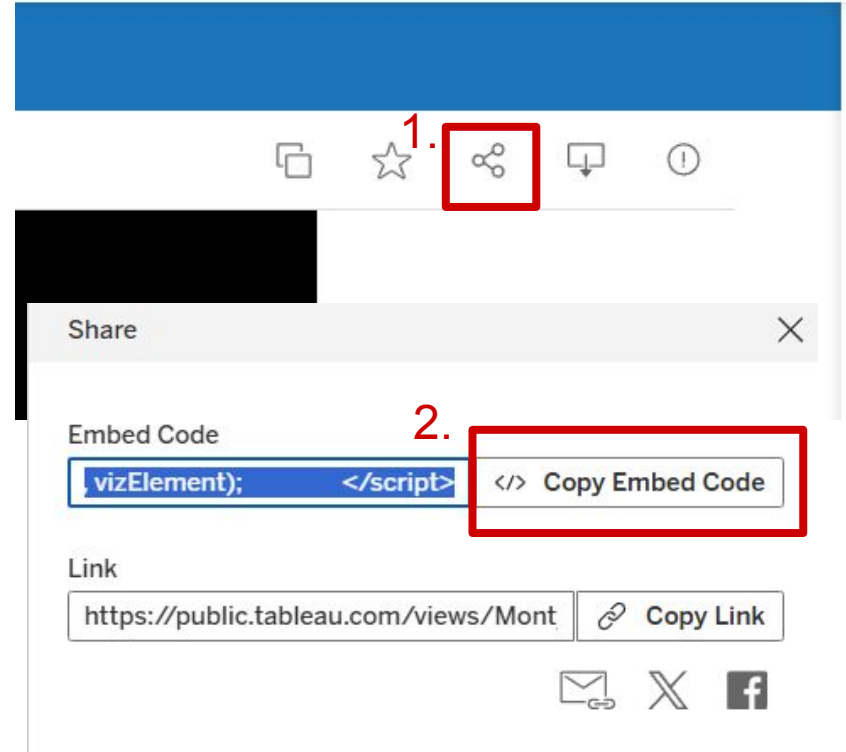
Sometimes doesn't work until you go live

YOUTUBE



4. Paste into your code

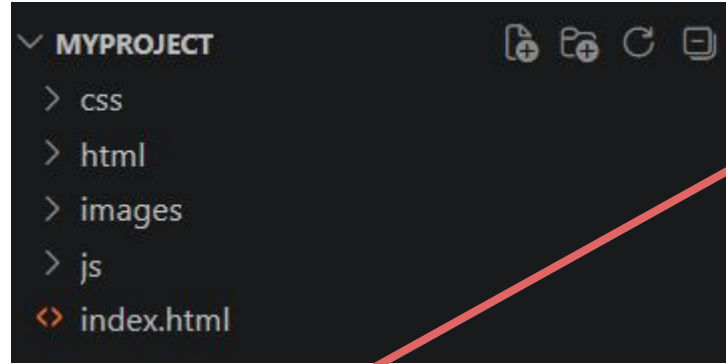
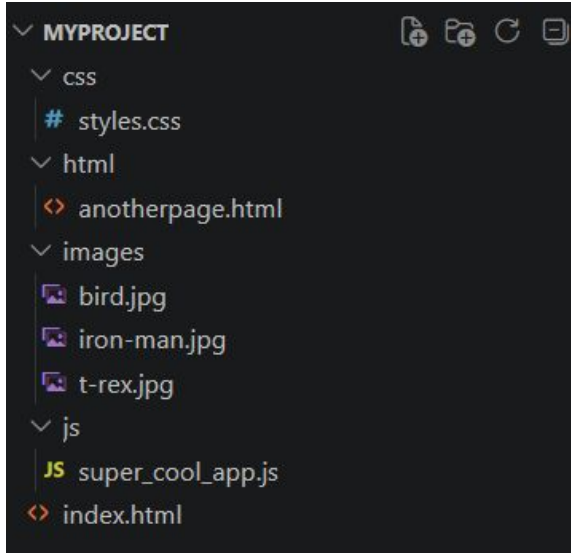
TABLEAU



3. Paste into your code

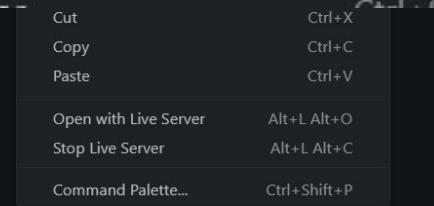
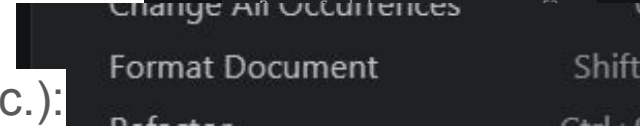
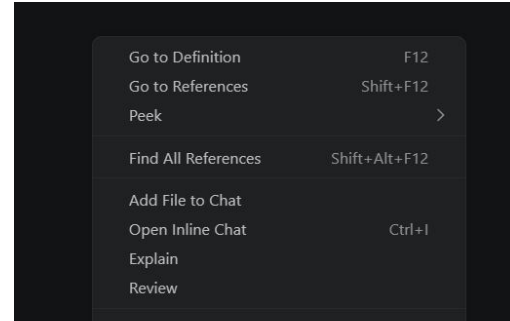
HTML: FOLDER STRUCTURE & FORMATTING

USE FOLDERS!

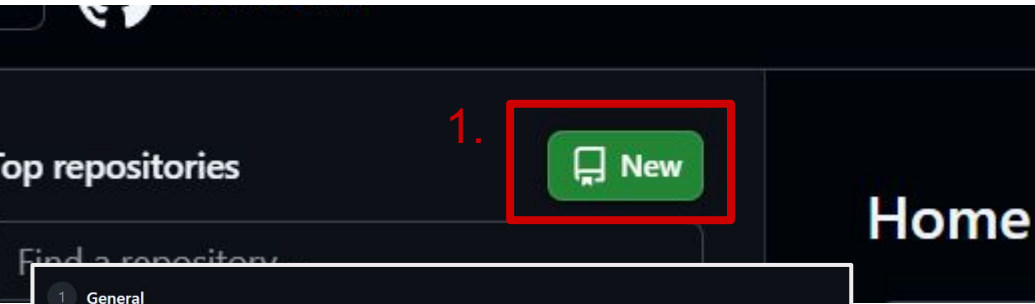


To quickly format everything correctly (tabs, long lines, etc.):

Right click > Format Document



HTML: UPLOADING TO GITHUB



1 General

Owner * jimna-h

Repository name * ta-example-website

Great repository names are short and memorable. How about [redesigned-happiness](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

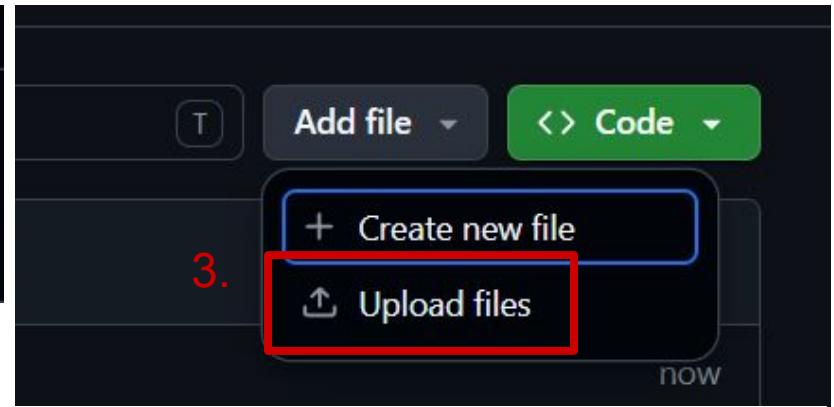
Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

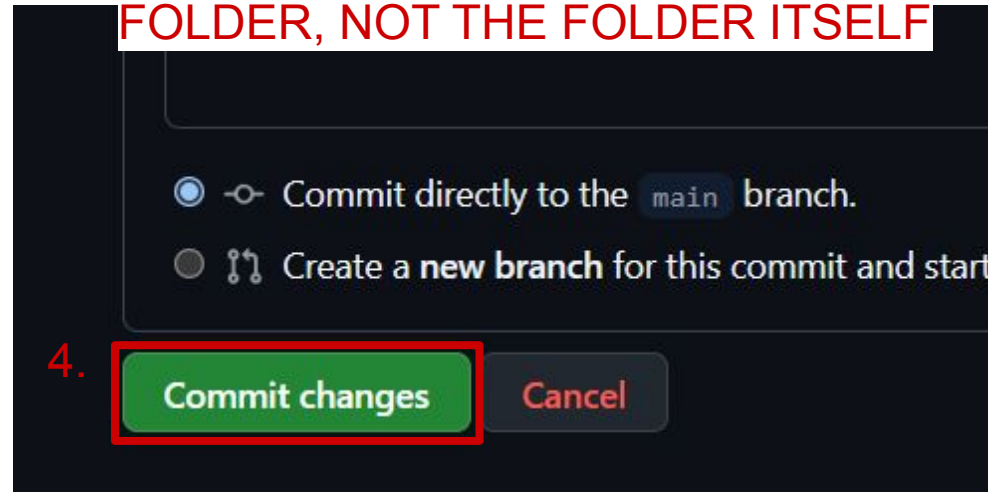
Add license

Licenses explain how others can use your code. [About licenses](#)

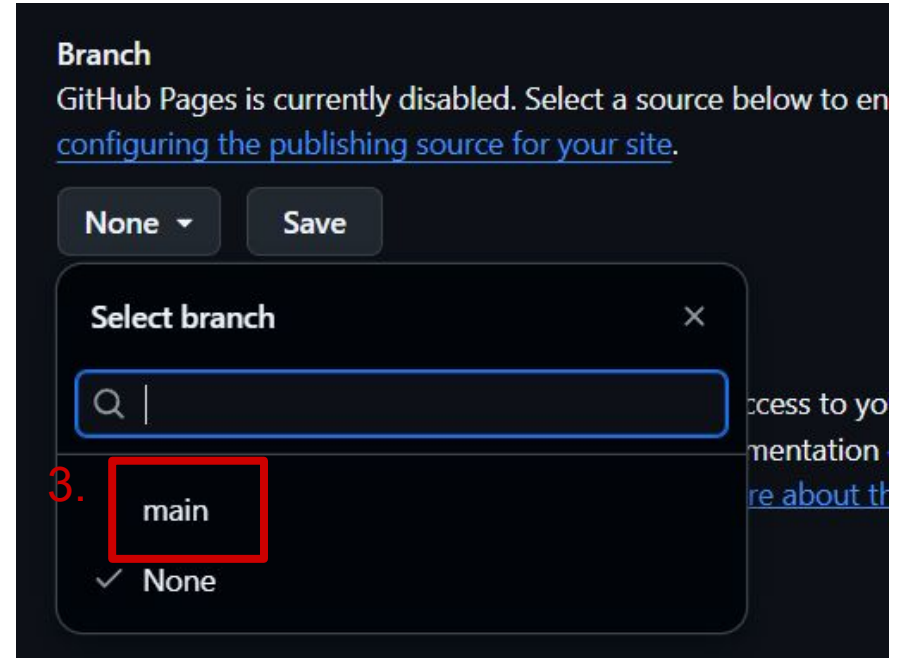
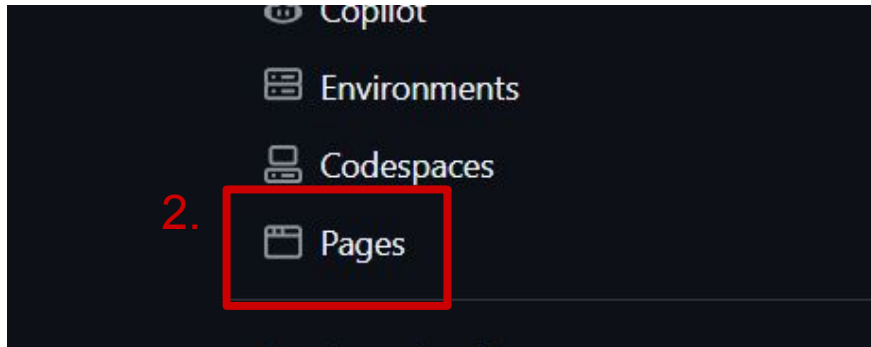
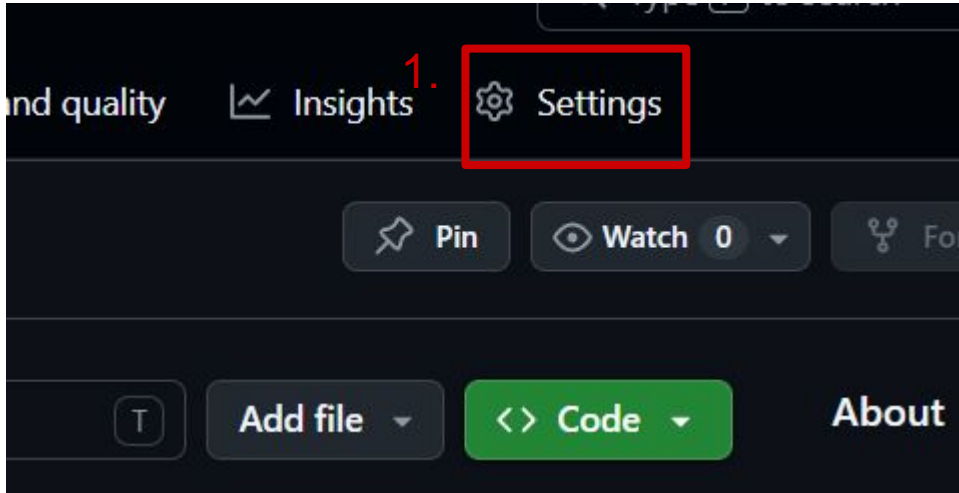
2. **Create repository**



UPLOAD THE CONTENTS OF YOUR FOLDER, NOT THE FOLDER ITSELF



HTML: GOING LIVE



AFTER A COUPLE OF MINUTES,
REFRESH THE PAGE AND THERE WILL
BE A LINK TO YOUR WEBSITE AT THE
TOP

CSS: BASICS

```
<!-- Connect CSS to HTML -->  
<link rel="stylesheet" href="styles.css" />
```

← In your HTML head

This is all CSS is →
(see next slide)

```
selector{  
    property: value;  
    property2: value2;  
}
```

CSS is **cascading** because more recent styling will supersede previous styling. If I tell the font color to be blue, and then I tell it to be red, it will be red.

Cascading
Style
Sheets

CSS: SELECTORS/PROPERTIES/VALUES

/*Built-in Selectors

These are all of the tags you use in your HTML document.

Think: body, h1, p, a, img, etc.

You can style them by using the tag name as the selector.*/

```
body{  
  background-color: ■ #f0f0f0;  
  font-family: Arial, sans-serif;  
  margin: 0;  
  padding: 20px;  
}
```

/*Class (custom) Selectors

These are selectors that you create yourself.

You can create a class by using a period, followed by the class name.*/

```
.highlight {  
  background-color: ■ yellow;  
  font-weight: bold;  
}
```

/*In HTML, you can apply this class to any element like this:*/

```
<p class="highlight">This text will be highlighted.</p>
```

There are A LOT of
selectors and
properties. Google
and AI are your
friend for finding
exactly what you
want!



bit.ly/4tZYPBK